Modern Computer Architecture • B.Tech (CS&AI)

Projects 1–12 | 12 Architecture-Oriented RTL Projects

This document presents twelve intermediate-level project options derived from the Modern Computer Architecture course outline. The projects progress from processor datapath and ISA implementation to pipelining, hazards, cache/memory systems, multicore processing, SIMD and AI-oriented systolic architectures. The course explicitly emphasizes implementation and verification of CPU components and memory systems using Verilog and simulation tools.

1. Course Alignment

The course outline covers digital foundations, sequential circuits and register files; MIPS/RISC-V organization and ISA; the five-stage IF–ID–EX–MEM–WB pipeline; RAW/WAW/WAR hazards, forwarding and branch prediction; cache organization, replacement and write policies; virtual memory; multicore/cache coherence; SIMD and modern AI hardware. These project topics are selected to give students practical Verilog implementation experience across those themes.

The syllabus also states that students should be able to create functional processor components including ALUs, memory systems and control units using hardware description languages, and translate theoretical concepts into verified hardware implementations.

2. Project List

No.	Project Title	Level	Primary Area
1	Single-Cycle RV32I Processor	Intermediate+	RISC-V / CPU Datapath
2	Single-Cycle MIPS Processor with Memory Interface	Intermediate+	MIPS / CPU Datapath
3	5-Stage Pipelined RISC-V Processor	Advanced Intermediate	Pipelining
4	Pipeline Hazard Detection and Forwarding Unit	Intermediate+	Hazards / Forwarding
5	Dynamic Branch Prediction Unit	Intermediate+	Branch Prediction
6	Direct-Mapped Cache Controller	Intermediate	Cache
7	2-Way Set-Associative Cache with LRU	Intermediate+	Cache / LRU
8	Write-Back Cache Controller	Advanced Intermediate	Write-Back Cache
9	Instruction and Data Memory Hierarchy	Intermediate	Memory Organization
10	Dual-Core Shared-Memory Processor	Advanced Intermediate	Multicore
11	SIMD Vector Processing Unit	Intermediate+	SIMD / Parallelism
12	Systolic Array Matrix Multiplication Accelerator	Advanced Intermediate	AI Hardware / Systolic Array

3. Detailed Project Specifications

Project 1: Single-Cycle RV32I Processor

Level: Intermediate+

Objective: Design and verify a compact single-cycle RISC-V processor in Verilog that executes a selected subset of RV32I instructions.

Architecture / Concepts: RISC-V ISA, fetch-decode-execute cycle, datapath, control unit, register file, ALU, instruction/data memory.

Suggested Verilog Modules: Program Counter (PC); Instruction Memory; Instruction Decoder/Control Unit; 32×32 Register File; Immediate Generator; ALU; Data Memory; Write-Back MUX.

Suggested Instruction Set: ADD, SUB, AND, OR, XOR, ADDI, LW, SW, BEQ and BNE.

Implementation Workflow: Develop the datapath and control signals; implement each RTL module; integrate the processor; create instruction-memory test programs; verify register and memory results; synthesize and, if available, deploy to an FPGA.

Expected Deliverables: Working RV32I processor, RTL/testbench, simulation waveforms, instruction test programs, synthesis/resource report and performance observations.

Evaluation Focus: Correct instruction execution, modular RTL, testbench coverage, waveform evidence, FPGA/synthesis results and explanation of datapath/control operation.

Project 2: Single-Cycle MIPS Processor with Memory Interface

Level: Intermediate+

Objective: Implement a basic 32-bit single-cycle MIPS processor capable of executing arithmetic, memory and branch instructions.

Architecture / Concepts: MIPS32 ISA, instruction formats, datapath, control signals, register file, ALU, memory interface and branching.

Suggested Verilog Modules: PC; Instruction Memory; MIPS Decoder; Register File; Sign Extension; ALU; Data Memory; Branch/Jump Logic; Write-Back Logic.

Suggested Instruction Set: R-type arithmetic/logic instructions, LW, SW, BEQ, ADDI and J.

Implementation Workflow: Define supported instruction formats; create the datapath; derive control signals; implement RTL modules; load small programs such as array sum, maximum search or factorial; verify execution.

Expected Deliverables: MIPS CPU RTL, testbench, waveform results, sample programs, synthesis report and optional FPGA demonstration.

Evaluation Focus: Instruction correctness, control-signal correctness, memory operation verification, program execution and design documentation.

Project 3: 5-Stage Pipelined RISC-V Processor

Level: Advanced Intermediate

Objective: Design a five-stage pipelined RISC-V processor and study the effect of pipelining on execution performance.

Architecture / Concepts: IF-ID-EX-MEM-WB pipeline, pipeline registers, CPI, IPC, throughput, datapath/control partitioning.

Suggested Verilog Modules: PC/IF Stage; IF/ID Register; Decode/Register File; ID/EX Register; ALU/EX Stage; EX/MEM Register; Memory/MEM Stage; MEM/WB Register; Write-Back; Control Unit.

Suggested Instruction Set: Use a manageable RV32I subset such as ADD, SUB, ADDI, LW, SW, BEQ and BNE.

Implementation Workflow: First verify the single-cycle datapath behavior; partition it into five stages; insert pipeline registers; verify instruction overlap; create dependent instruction sequences; measure cycles and CPI.

Expected Deliverables: Five-stage pipelined CPU, pipeline timing/waveforms, benchmark programs, CPI/IPC comparison and synthesis/FPGA results.

Evaluation Focus: Correct stage operation, pipeline-register design, instruction ordering, performance analysis and verification quality.

Project 4: Pipeline Hazard Detection and Forwarding Unit

Level: Intermediate+

Objective: Develop a hardware unit that detects data hazards in a pipelined processor and resolves them using forwarding and controlled stalling.

Architecture / Concepts: RAW hazards, load-use hazards, forwarding paths, pipeline stalls, hazard-control signals.

Suggested Verilog Modules: Register-ID Comparators; Forwarding Unit; Hazard Detection Unit; Forwarding MUX Control; Stall/Flush Logic; Testbench.

Suggested Instruction Set: Use representative ALU dependencies and load-use sequences such as ADD followed by SUB, and LW followed by a dependent instruction.

Implementation Workflow: Generate hazard cases; identify source/destination register dependencies; implement EX/MEM and MEM/WB forwarding; add load-use stall logic; integrate with a five-stage pipeline; compare execution with and without hazard handling.

Expected Deliverables: Hazard/forwarding RTL, verification testbench, waveform cases, CPI comparison and integration report.

Evaluation Focus: Correct detection of dependencies, correct forwarding selection, correct stall behavior and measurable performance improvement.

Project 5: Dynamic Branch Prediction Unit

Level: Intermediate+

Objective: Design and evaluate a hardware branch-prediction unit for reducing control-hazard penalties in a pipelined processor.

Architecture / Concepts: Control hazards, branch target prediction, prediction accuracy, misprediction penalty, 1-bit and 2-bit prediction.

Suggested Verilog Modules: Branch History Table; 1-bit/2-bit Predictor; Prediction Logic; Update Logic; Branch Target Storage/Logic; Pipeline Flush/Control Interface.

Suggested Instruction Set: Evaluate conditional branch instructions such as BEQ and BNE using loop-oriented instruction traces.

Implementation Workflow: Implement an always-not-taken baseline; add a 1-bit predictor; extend to a 2-bit saturating predictor; feed branch traces; calculate prediction accuracy and estimate CPI impact.

Expected Deliverables: Predictor RTL, branch traces, accuracy tables, waveform verification and comparison of prediction strategies.

Evaluation Focus: Correct state transitions, predictor update behavior, accuracy analysis and clear performance comparison.

Project 6: Direct-Mapped Cache Controller

Level: Intermediate

Objective: Design a direct-mapped cache controller and analyze cache hit/miss behavior using representative memory-access traces.

Architecture / Concepts: Memory hierarchy, temporal/spatial locality, tag/index/offset, cache hit/miss, hit ratio, miss ratio and AMAT.

Suggested Verilog Modules: Cache Data RAM; Tag RAM; Valid-Bit RAM; Address Decoder; Tag Comparator; Cache Controller FSM; Main-Memory Interface.

Suggested Parameters: A practical student configuration can use a small cache such as 1 KB with 16-byte blocks and 32-bit addresses; students may vary these parameters.

Implementation Workflow: Derive tag/index/offset fields; implement cache storage; implement lookup and refill; generate sequential, random and repeated accesses; calculate hit rate and AMAT.

Expected Deliverables: Cache RTL, controller FSM, memory-access trace, hit/miss report, AMAT calculation and synthesis results.

Evaluation Focus: Correct cache lookup/refill, valid-bit handling, hit/miss statistics and architectural analysis.

Project 7: 2-Way Set-Associative Cache with LRU

Level: Intermediate+

Objective: Implement a two-way set-associative cache with an LRU replacement mechanism and compare it against a direct-mapped cache.

Architecture / Concepts: Set associativity, tag comparison, replacement policy, LRU, locality and cache performance.

Suggested Verilog Modules: Way-0 Data/Tag RAM; Way-1 Data/Tag RAM; Valid Bits; LRU State; Tag Comparators; Cache FSM; Memory Interface.

Suggested Parameters: Choose a small cache configuration suitable for FPGA simulation and synthesis; document tag, index and offset widths.

Implementation Workflow: Implement two cache ways; perform parallel tag comparison; select hit way; update LRU state; handle misses and replacement; run identical traces on direct-mapped and two-way caches.

Expected Deliverables: Two-way cache RTL, LRU logic, access traces, hit-rate comparison and resource/performance report.

Evaluation Focus: Correct way selection, LRU update, replacement behavior, comparison methodology and verification.

Project 8: Write-Back Cache Controller

Level: Advanced Intermediate

Objective: Design a write-back cache that reduces main-memory traffic using dirty blocks and controlled write-back on replacement.

Architecture / Concepts: Write-back policy, dirty bits, write hit/miss, eviction, refill, cache-controller FSM.

Suggested Verilog Modules: Cache Data RAM; Tag RAM; Valid RAM; Dirty-Bit RAM; Tag Comparator; Write-Back FSM; Memory Interface.

Implementation Workflow: Implement read hits and misses; implement write hits using dirty bits; handle dirty-block eviction; write the old block to memory before refill; verify all controller states.

Expected Deliverables: Write-back cache RTL, FSM/state diagram, directed testbench, waveform verification and memory-traffic comparison.

Evaluation Focus: Correct dirty-bit management, eviction/refill sequencing, write correctness and comparison with write-through behavior.

Project 9: Instruction and Data Memory Hierarchy

Level: Intermediate

Objective: Implement a simplified Harvard-style memory organization with separate instruction and data paths and analyze its behavior.

Architecture / Concepts: Von Neumann vs Harvard architecture, instruction memory, data memory, address decoding and memory organization.

Suggested Verilog Modules: Program Counter; Instruction Memory; Data Memory; Address Decoder; Read/Write Control; Optional Shared Lower-Level Memory.

Implementation Workflow: Create independent instruction and data paths; connect them to a simple processor or controller; execute programs containing instruction fetches and data accesses; analyze simultaneous access opportunities.

Expected Deliverables: Harvard memory architecture RTL, block diagram, simulation waveforms and comparison with a shared-memory organization.

Evaluation Focus: Correct instruction/data separation, memory control, timing behavior and architecture comparison.

Project 10: Dual-Core Shared-Memory Processor

Level: Advanced Intermediate

Objective: Build a simplified dual-core processor system in which two processor cores access a shared memory through an arbitration mechanism.

Architecture / Concepts: Multicore architecture, shared memory, parallel execution, memory arbitration and throughput.

Suggested Verilog Modules: Core 0; Core 1; Shared RAM; Memory Arbiter; Address/Data Multiplexers; Request/Grant Logic; Synchronization/Status Logic.

Implementation Workflow: Use two simple processor cores or two instruction engines; assign independent workloads; implement shared-memory arbitration; run workloads in parallel; compare against single-core execution.

Expected Deliverables: Dual-core RTL, arbiter FSM, workload programs, simulation waveforms, execution-cycle comparison and synthesis report.

Evaluation Focus: Correct arbitration, absence of unintended simultaneous writes, workload partitioning and performance analysis.

Project 11: SIMD Vector Processing Unit

Level: Intermediate+

Objective: Design a small SIMD unit that performs multiple identical arithmetic/logic operations in parallel.

Architecture / Concepts: Data-level parallelism, SIMD, vector registers, parallel ALUs, throughput and speed-up.

Suggested Verilog Modules: Vector Register File; Multiple ALUs; Vector Control Unit; Lane MUXes; Result Combiner; Optional Scalar Interface.

Suggested Operations: Vector ADD, SUB, AND, OR and comparison; optionally include shifts or multiply.

Implementation Workflow: Implement four or eight lanes; create scalar and SIMD versions of the same workload; verify vector results; measure cycle count, throughput and FPGA resource usage.

Expected Deliverables: SIMD RTL, vector testbench, scalar-vs-SIMD timing comparison, resource utilization and maximum-frequency results.

Evaluation Focus: Correct lane operation, parallel execution, verification coverage and quantitative performance analysis.

Project 12: Systolic Array Matrix Multiplication Accelerator

Level: Advanced Intermediate

Objective: Design a small systolic-array accelerator for matrix multiplication using interconnected processing elements.

Architecture / Concepts: Parallel processing, processing elements, MAC operation, systolic data movement, throughput and AI acceleration.

Suggested Verilog Modules: Processing Element (PE); MAC Unit; Input Buffers; Output/Accumulator Registers; Systolic Array Controller; Matrix Data Interface.

Suggested Parameters: A 3×3 or 4×4 array is appropriate for a B.Tech project. Use fixed-point operands to keep FPGA implementation practical.

Implementation Workflow: Design one PE; verify multiply-accumulate behavior; connect PEs into an array; stream matrix elements through the array; verify matrix multiplication against a reference model; analyze latency and throughput.

Expected Deliverables: Systolic-array RTL, PE testbench, matrix-multiplication demonstration, cycle/throughput analysis and FPGA resource report.

Evaluation Focus: Correct data propagation, MAC accuracy, matrix result, timing analysis and comparison with sequential multiplication.

4. Common Project Requirements

- System/architecture block diagram and explanation.
- Microarchitecture or state-transition description where applicable.
- Synthesizable Verilog RTL with modular hierarchy.
- Dedicated Verilog testbench with meaningful directed test cases.
- Functional simulation and waveform verification using a simulator/GTKWave or equivalent.
- At least one quantitative performance metric such as CPI, hit rate, throughput, latency or speed-up.
- Synthesis results including resource utilization and maximum operating frequency .
- Comparison against a baseline architecture or alternative design wherever applicable.
- Final technical report and live demonstration.

5. Suggested FPGA / Tool Flow

A practical implementation flow is: Verilog RTL → simulation/testbench → waveform verification → synthesis → timing/resource analysis → FPGA implementation (where available). Suitable tools may include Vivado for Xilinx FPGA boards, ModelSim/Questa or other Verilog simulators, and GTKWave for waveform inspection.